



Faculty of Engineering and Technology
Electrical and Computer Engineering Department
Second Semester 2025-2026
ENCS3340
ARTIFICIAL INTELLIGENCE
Project#2

Prepared by:

Kareen Kamel-1230509 Section:4

Instructor: Dr. Yazan Abu Farha

Introduction

The objective of this project is to gain practical experience in implementing and evaluating machine learning classification algorithms using the Scikit-learn library in Python. The project focuses on supervised learning, where the training data is labeled, meaning that the correct class of each example is already known. The learning algorithm uses these labeled examples to learn the relationship between the input features and the target class, allowing it to predict the class of new unseen instances.

Two supervised learning algorithms were studied in this project: Decision Tree and K-Nearest Neighbors (KNN). A Decision Tree builds a tree-like model by repeatedly selecting the attribute with the highest information gain (or the greatest reduction in entropy) to split the data into smaller and purer groups until a classification decision is reached. In contrast, KNN is an instance-based learning algorithm that does not build a model during training. Instead, when a new sample is presented, it finds the K nearest training samples based on their distance and assigns the class that appears most frequently among those neighbors.

Since the PlayTennis dataset contains only 14 instances, Leave-One-Out Cross Validation (LOOCV) was used to obtain a more reliable evaluation of the classifiers. In LOOCV, one instance is used as the test sample while the remaining instances are used for training, and the process is repeated until every instance has been tested once.

Finally, the performance of the classifiers was evaluated using several performance metrics, including Accuracy, Precision, Recall, and F1-score. In addition, a Confusion Matrix was used to visualize the classification results and analyze the correctly and incorrectly classified instances.

Part A: Decision Tree Classification

Play Tennis Dataset Description

The PlayTennis dataset consists of 14 instances and 4 input features: Outlook, Temperature, Humidity, and Wind. The target variable is PlayTennis, which has two possible classes (Yes or No), making it a binary classification problem. This dataset was used to train and evaluate the Decision Tree classifier.

Dataset Encoding

Before training the classifier, all categorical features were converted into numerical representations using the LabelEncoder technique provided by Scikit-learn. This step ensured that the machine learning algorithm could interpret and process the dataset correctly. The same encoding process was applied to the target variable to represent the class labels numerically. For example, Outlook was encoded as Overcast = 0, Rain = 1, and Sunny = 2. The target labels were also encoded as No = 0 and Yes = 1.

Decision Tree Implementation

For the Decision Tree training, the Scikit-learn library was used to build and train the classifier. The tree was constructed using entropy and information gain, where the algorithm selects the attributes that reduce uncertainty and produce the purest possible splits, improving the classification performance. Since the PlayTennis dataset is relatively small, Leave-One-Out Cross Validation (LOOCV) was used for evaluation. In this approach, one instance is used for testing while the remaining instances are used for training, and the process is repeated for all instances in the dataset. This provides a more reliable estimate of the model's performance on unseen data.

Results

The Decision Tree classifier correctly classified 8 out of 14 instances in the PlayTennis dataset. The predicted labels matched the actual labels for most of the samples, resulting in an accuracy of 57.14% using Leave-One-Out Cross Validation (LOOCV).

```
Leave-One-Out Cross Validation Results:
-----
Test 1: Actual = 0, Predicted = 0 --> Correct
Test 2: Actual = 0, Predicted = 0 --> Correct
Test 3: Actual = 1, Predicted = 1 --> Correct
Test 4: Actual = 1, Predicted = 0 --> Wrong
Test 5: Actual = 1, Predicted = 0 --> Wrong
Test 6: Actual = 0, Predicted = 1 --> Wrong
Test 7: Actual = 1, Predicted = 1 --> Correct
Test 8: Actual = 0, Predicted = 0 --> Correct
Test 9: Actual = 1, Predicted = 1 --> Correct
Test 10: Actual = 1, Predicted = 1 --> Correct
Test 11: Actual = 1, Predicted = 0 --> Wrong
Test 12: Actual = 1, Predicted = 0 --> Wrong
Test 13: Actual = 1, Predicted = 1 --> Correct
Test 14: Actual = 0, Predicted = 1 --> Wrong
-----
Correct Predictions: 8 out of 14
Decision Tree LOOCV Accuracy: 0.5714285714285714
```

Figure 1:Part A results

Tree Visualization

The tree was generated using all 14 instances of the PlayTennis dataset.

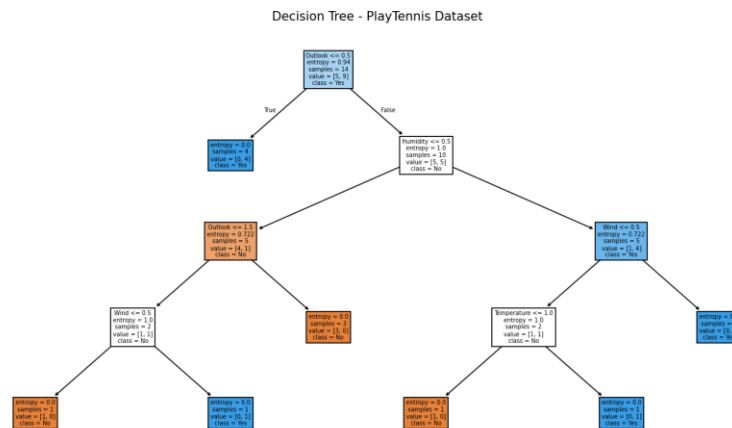


Figure 2: Generated Decision Tree for the PlayTennis Dataset

Decision Tree Interpretation

The generated Decision Tree selected Outlook as the root node, indicating that it provided the highest Information Gain among all attributes and it reduced the dataset entropy more than the other attributes and therefore produced the best split. The tree then used additional attributes such as Humidity and Wind to further classify the instances. Leaf nodes have an entropy value of 0, which means that all samples within those nodes belong to the same class and are classified with complete certainty. Overall, the Decision Tree was able to learn clear decision rules from the dataset and effectively distinguish between the "Yes" and "No" classes.

Part B: Classifier Comparison(KNN)

Methodology

For this part, the same PlayTennis dataset was used. A K-Nearest Neighbors (KNN) classifier was implemented using the KNeighborsClassifier class from the Scikit-learn library. The value of K was set to 3, meaning that each classification decision was based on the three nearest neighbors. The model was trained using the encoded dataset and evaluated using Leave-One-Out Cross Validation (LOOCV).

Results

```

KNN Leave-One-Out Cross Validation Results:
Test 1: Actual = 0, Predicted = 0 --> Correct
Test 2: Actual = 0, Predicted = 0 --> Correct
Test 3: Actual = 1, Predicted = 1 --> Correct
Test 4: Actual = 1, Predicted = 0 --> Wrong
Test 5: Actual = 1, Predicted = 1 --> Correct
Test 6: Actual = 0, Predicted = 1 --> Wrong
Test 7: Actual = 1, Predicted = 1 --> Correct
Test 8: Actual = 0, Predicted = 0 --> Correct
Test 9: Actual = 1, Predicted = 0 --> Wrong
Test 10: Actual = 1, Predicted = 1 --> Correct
Test 11: Actual = 1, Predicted = 0 --> Wrong
Test 12: Actual = 1, Predicted = 1 --> Correct
Test 13: Actual = 1, Predicted = 1 --> Correct
Test 14: Actual = 0, Predicted = 1 --> Wrong

```

Figure 4:part B KNN Results

Classification Report:				
	precision	recall	f1-score	support
No	0.50	0.60	0.55	5
Yes	0.75	0.67	0.71	9
accuracy			0.64	14
macro avg	0.62	0.63	0.63	14
weighted avg	0.66	0.64	0.65	14

Figure 3:KNN classification report

The classifier correctly classified 3 instances of the "No" class and 6 instances of the "Yes" class. However, 2 "No" instances were incorrectly classified as "Yes", and 3 "Yes" instances were incorrectly classified as "No". Overall, the confusion matrix shows that the KNN classifier correctly classified 9 out of 14 instances, resulting in an accuracy of 64.29%.

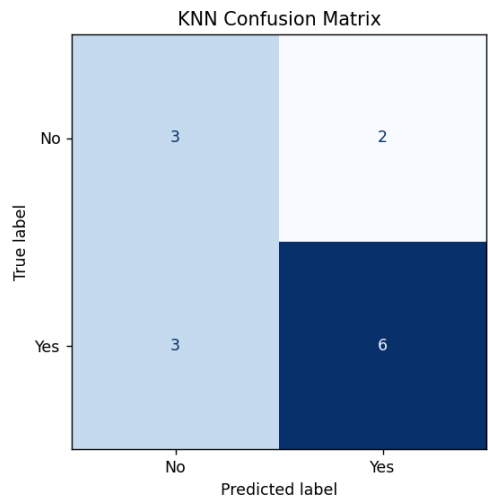


Figure 5:KNN Confusion Matrix

Comparison

Metric	Decision Tree	KNN K=3
Accuracy	0.57	0.64
Precision (No)	0.43	0.50
Precision (Yes)	0.71	0.75
Recall (No)	0.60	0.60
Recall (Yes)	0.56	0.67
F1-Score (No)	0.50	0.55
F1-Score (Yes)	0.62	0.71

Table 1: Decision Tree and KNN Results Comparison

Interpretation

The results shown in Table 1 indicate that the KNN classifier outperformed the Decision Tree on the PlayTennis dataset under LOOCV evaluation. KNN achieved an accuracy of 64% compared to 57% for the Decision Tree. Similarly, KNN showed higher Precision, Recall, and F1-Score across both classes. For the Yes class, KNN achieved an F1-Score of 0.71 compared to 0.62 for the Decision Tree. For the No class, KNN achieved an F1-Score of 0.55 compared to 0.50. The relatively low accuracy of both classifiers is expected due to the very small size of the PlayTennis dataset, which contains only 14 instances. With such limited data, both models struggle to learn robust decision boundaries, and LOOCV makes the evaluation stricter by training on only 13 instances at a time.

Part C: Classification on a UCI Dataset (Iris Dataset)

Iris Dataset Description

For this part, the Iris dataset was selected from the UCI Machine Learning Repository. The dataset is commonly used for classification tasks and contains measurements of iris flowers. The objective is to classify each flower into one of three species: Setosa, Versicolor, or Virginica.

The dataset consists of 150 instances and 4 numerical features: sepal length, sepal width, petal length, and petal width. These features are used as input variables, while the flower species serves as the target variable. Since the dataset contains three possible classes, this is considered a multiclass classification problem.

Methodology

In this part, the K-Nearest Neighbors (KNN) classifier was used with a value of $K = 3$. The Iris dataset was divided into 80% training data and 20% testing data. The model was first trained using the training set and then evaluated on the testing set. This approach allows the classifier to learn from one portion of the data and assess its performance on previously unseen samples.

Results

Using the `classification_report` function from `sklearn.metrics`, the evaluation metrics Precision, Recall, F1-score, and Support were calculated. In addition, the model achieved an accuracy of 1.00 (100%), indicating that all test samples were classified correctly. Since the model produced no misclassifications, the Precision, Recall, and F1-score values were equal to 1.00 for all classes, demonstrating perfect classification performance. The Support value represents the number of test instances belonging to each class. In this experiment, the test set contained 10 Setosa samples, 9 Versicolor samples, and 11 Virginica samples, resulting in a total of 30 test instances.

Accuracy: 1.0				
	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	9
2	1.00	1.00	1.00	11

Figure 6:Accuracy, Precision, Recall, F1-Score, and Support Values Obtained from the KNN Classifier on the Iris Dataset

Confusion Matrix

The confusion matrix shows that all test samples were classified correctly by the KNN classifier. The model correctly identified 10 Setosa samples, 9 Versicolor samples, and 11 Virginica samples, with no misclassifications. Since all values outside the main diagonal are zero, the classifier achieved perfect classification performance on the test set.

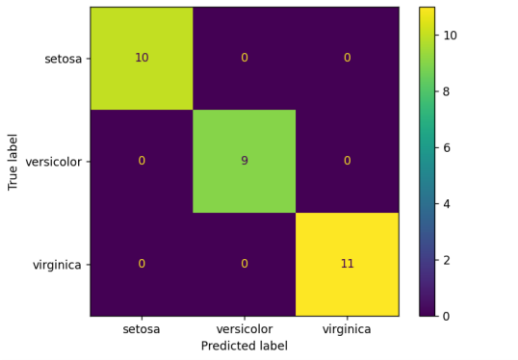


Figure 7:Confusion Matrix

Observations

The results show that the KNN classifier was able to correctly classify all test samples in the Iris dataset, achieving an accuracy of 100%. The Precision, Recall, and F1-score values were all equal to 1.00, indicating excellent classification performance. These results suggest that the Iris dataset is well-structured and that its classes are clearly separable. One of the main strengths of KNN is its simplicity and its ability to achieve high classification accuracy when the classes are well separated. However, KNN requires calculating the distance between a new sample and all training samples during prediction, which can increase the prediction time for very large datasets. In this project, the Iris dataset was relatively small and well organized, allowing KNN to classify all instances correctly and effectively distinguish between the three Iris species.

Conclusion

After implementing different machine learning classification techniques on the same dataset, we concluded that the K-Nearest Neighbors (KNN) classifier achieved better performance than the Decision Tree classifier. KNN obtained higher Accuracy, Precision, Recall, and F1-score values, while the Decision Tree produced lower performance on the PlayTennis dataset. However, the Decision Tree provided a clear and easy-to-understand visualization of the classification process, making it easier to interpret how the final decision was reached. This difference in performance is likely due to the small size of the dataset, which contains only 14 instances and was evaluated using Leave-One-Out Cross Validation .

The KNN classifier was also applied to a different dataset, the Iris dataset, which is larger and contains three classes. In this case, the classifier achieved excellent performance with 100% accuracy and perfect Precision, Recall, and F1-score values. This demonstrates that the performance of a machine learning algorithm depends not only on the algorithm itself but also on the characteristics and size of the dataset.

Overall, this project provided practical experience in implementing supervised learning algorithms, evaluating their performance using different metrics, and comparing their effectiveness on different datasets.